Graph-Structured Models, Their Algorithms, and the Properties That Referee Them

M. J. Wilson

September 8, 2026

Abstract

A self-contained account of the three problem classes this project supports, the algorithms that solve them, and—the part a methods section usually omits—the mathematical properties each algorithm is validated against, and why those properties are the right referees. Written for a reader with a scientific and performance-computing background but no background in phylogenetics or statistical physics. It describes *algorithms*, not an implementation: no result here depends on how any of it is coded, and the companion paper carries the results themselves.

Contents

1 Notation

One symbol, one meaning, across this document and the companion paper.

Symbol	Meaning
$\mathcal{T}$	a tree topology
$\mathcal{G}$	a general undirected graph
$\mathcal{A}$	the state alphabet, $\lvert\mathcal{A}\rvert = k$
ρ	the root of a rooted tree
$\mathcal{N}(\cdot)$	the neighbourhood a discrete move set reaches in one step
$\mathbf{Q}$	a rate matrix; $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ its transition matrix
$\mathbf{S}$	a symmetric exchangeability matrix
$\boldsymbol{\pi}$	a stationary or initial distribution
$\boldsymbol{\tau}$	branch lengths
$\boldsymbol{\theta}$	unconstrained parameters, $\boldsymbol{\theta} \in \mathbb{R}^d$
ϕ	the constraint map, $\phi(\boldsymbol{\theta})$ the feasible parameters
$\mathbf{D}$	the observed data
$\mathcal{L}$	the scalar being minimized
H	a Hamiltonian
$\mathbf{Z}$	a hidden state path
$\mathcal{E}$	an emission family
x_i	a discrete variable of a factor graph, $x_i \in \mathcal{A}$
ψ_a	a factor: a non-negative function of the variables $x_{\partial a}$ it touches
$m_{a\to i},\ m_{i\to a}$	a message from factor to variable, and from variable to factor
$b_i,\ b_a$	a belief: the marginal of a variable or a factor, exact on a tree
Z	a partition function; $\log Z$ the evidence
$T,\ \beta = 1/T$	a temperature and its inverse; a schedule is $T(t)$ over a budget of t steps

Two conventions are worth stating because they are load-bearing later. Optimization is always *minimization*, so a likelihood-based objective is a negative log-likelihood. And $\mathcal{L}$ is a function of $\boldsymbol{\theta}$, the unconstrained vector, not of the parameters a person names—the two are connected by ϕ and Section 2.2 explains why the distinction is not cosmetic.

A third convention governs the symbols the problem sections keep. Every model below is an instance of the factor graph of Section 2.1, and its variables are the x_i of that section; but a spin is written σ_i and a hidden state z_t, because those are the names a reader arrives with. The identification is made once, where each model is stated as a factor graph, and not repeated: $x_i \equiv \sigma_i$ on a Potts graph, $x_t \equiv z_t$ on a chain, $x_u \equiv z_u$ at a node of a tree. Every section then carries the same four parts—the model, the model as a factor graph, the algorithm as the instance of Equation 2 or of the optimization it is, and the property that pins it—so that what differs between the problems is visible as content and not as presentation.

2 The Shape All Four Problems Share

Each problem class below couples a *combinatorial* search over a structure to a *continuous* optimization of parameters given that structure. The two are not interleaved steps of one optimizer: a structural move changes what the parameter vector *means* and how long it is, so it constructs a new objective rather than taking a step inside an existing one. Everything else in this document follows from that division.

2.1 Factor Graphs: The One Structure

A tree under a substitution model, a Potts model on a graph and a hidden Markov chain are one object: a set of discrete variables $x_1, \ldots, x_N$ and a set of factors ψ_a, each a non-negative

function of a subset $x_{\partial a}$, with

$$p(x) = \frac{1}{Z} \prod_a \psi_a(x_{\partial a}), \qquad Z = \sum_x \prod_a \psi_a(x_{\partial a}). \tag{1}$$

The bipartite graph between variables and factors is the factor graph [23, 22]. In its *normal* (Forney) form every variable is an edge between exactly two factors, a variable shared by more factors passing through an equality factor $\delta(x, x', \dots)$ [13]; the form changes the drawing and not the distribution, which is checked by computing the same marginals on both. Observations enter as factors: a leaf's state is an indicator, an emission is a unary factor over the hidden state, so the graph is over latent variables only and a density and a probability are handled alike.

Sum-product sends, from factor a to variable i,

$$m_{a\to i}(x_i) = \sum_{x_{\partial a \setminus i}} \psi_a(x_{\partial a}) \prod_{j \in \partial a \setminus i} m_{j\to a}(x_j), \qquad m_{i\to a}(x_i) = \prod_{b \in \partial i \setminus a} m_{b\to i}(x_i), \tag{2}$$

and on a tree one leaf-to-root and one root-to-leaf pass are exact: on the tree's factor graph this is pruning, Equation 14; on a chain it is the forward recursion, Equation 25; on a pairwise graph it is Equation 17. On a loopy graph the fixed point is the Bethe approximation of Equation 18. Replacing the sum by a maximum gives max-product, whose max-marginals carry the MAP assignment—on a chain, Viterbi. One implementation of Equation 2 therefore serves all three problem classes, and each is pinned to the evaluator that predates it.

A fourth shape. The coupled spatio-sequential model puts a Potts prior on spatial labels $\ell_n \in \{1..M\}$, one hidden chain $k_{s,m}$ per class, and a *gated* emission joining one label to one chain state:

$$\log p(\ell, k, x) = \beta \sum_{\langle n,n' \rangle} J_{nn'}\, \delta(\ell_n, \ell_{n'}) + \sum_m \Big[\log \pi_m(k_{1,m}) + \sum_{s>1} \log A_m(k_{s-1,m}, k_{s,m}) \Big] + \sum_n \sum_s \log P(x_{sn} \mid k_{s,\ell_n}, \theta_{\ell_n} \tag{3}$$

which is Equation 1 with pairwise, chain and degree-two gated factors, and no fourth code path. Section 6 states the model in full, with the estimator its structure dictates.

A parity-check code. A low-density parity-check code is Equation 1 with binary variables, a unary factor per bit carrying the channel's log-likelihood ratio, and a hard parity constraint per row of the check matrix (Section 7). It is the problem message passing was invented for [15], and the fourth problem class: its decoder is Equation 2 specialised to one number per message, and is held to the general implementation on codes small enough for both.

What pins the general algorithm. Equation 2 is validated where each of its instances already has an oracle, and by nothing new: on a tree-shaped Potts graph against enumeration over $q^{|V|}$ configurations, on a chain against the sum over k^T paths, on a tree at one site against pruning, and on a loopy graph against the pairwise belief propagation of Equation 17—the same approximation reached by two codes, which is agreement and not truth. The Forney form is pinned by computing the same marginals on both drawings. A general algorithm that agreed only with itself would be a second opinion, not an evaluator (Section 8).

2.2 The Continuous Optimization Abstraction

Let the objective be a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with a map ϕ carrying $\boldsymbol{\theta}$ to the constrained parameters a model is stated in: positive branch lengths through a logarithm, a distribution over k states through a softmax on $k-1$ free values, a positive scale through a logarithm.

Why the map and not a projection. An optimizer that must be *stopped* from leaving the feasible set will eventually leave it, and a constrained parameter that reaches its boundary has no interval around it worth reporting. Constraining by construction removes both failures: every point in $\mathbb{R}^d$ is feasible, and the gradient $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ is defined everywhere.

Why the map must be injective. Where a reparameterization leaves $\mathcal{L}$ unchanged, the direction along it is not estimable: the observed information is singular and an interval is undefined. A softmax on k free values has exactly this flat direction—adding a constant to every value changes nothing—which is why $k-1$ are used. Removing the freedom is the constraint map's job, not the optimizer's.

What follows for inference. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method: $\mathrm{Var}(\phi(\boldsymbol{\theta})) \approx J\Sigma J^\top$ with J the Jacobian of ϕ. This is an *asymptotic* statement, exact only in the large-sample limit, which is why Section 8 makes realized interval coverage a measurement rather than an assumption.

2.3 The Discrete Search as a Decision Process

The structural search is a Markov decision process [37]: the state is the current structure with its fitted parameters, an action is a move from the neighbourhood $\mathcal{N}(\cdot)$, and the reward is the improvement in the objective. An episode ends on a step budget or when no move improves the score—a property of the state, and the same stopping rule the greedy baseline obeys, which is what makes the two comparable. Because the reward is a difference, the undiscounted return telescopes,

$$G = \sum_{t=0}^{T-1} R_t = \sum_{t=0}^{T-1} \big[\mathcal{L}(s_t) - \mathcal{L}(s_{t+1})\big] = \mathcal{L}(s_0) - \mathcal{L}(s_T), \tag{4}$$

the improvement between the first and last state whatever path joined them. That is what licenses an undiscounted return—a discount would prefer improvement found early over the same improvement found late—and it is why a truncation landing between a sacrifice and its payoff teaches the opposite of the truth, so a truncated episode is recorded as one.

The policy is a softmax over the scores of the *available* actions, $\pi(a \mid s) \propto \exp f(s,a)$ for $a \in \mathcal{N}(s)$, so a neighbourhood of any size is admissible. A scorer with a single weight on the improvement a move buys is an inverse temperature, and greedy search is its zero-temperature limit; what such an agent can learn is exactly how much it should accept a worsening move to reach a better optimum, which is the credit-assignment question this search makes sharp. It is trained by the score-function estimator with a baseline,

$$\nabla J = \mathbb{E}\Big[\sum_t \nabla \log \pi(a_t \mid s_t)\,\big(G_t - b(s_t)\big)\Big], \qquad G_t = \sum_{u \ge t} R_u, \tag{5}$$

where a baseline b independent of the sample it is subtracted from leaves the estimator unbiased and is there to reduce its variance, not its bias: G_t is a sum of objective differences whose scale is the problem's, so an uncentred gradient varies by orders of magnitude between instances. The comparison against the greedy baseline is at a matched budget of objective evaluations per decision, never in wall-clock time.

What pins it. Equation 4 is an identity and is asserted exactly on every recorded episode. Equation 5 is an expectation, so its estimator is pinned against the gradient of the exact expected return, which is computable by enumerating every episode on a small environment, and the baseline's unbiasedness is asserted by the same enumeration: the expected gradient with and without b agree. An environment small enough to enumerate is therefore the oracle for the learning machinery, exactly as a tree small enough to enumerate is the oracle for the search.

2.4 Surrogates and Bounds

The evaluation a discrete search pays for—a fit of $\boldsymbol{\tau}$ per topology, or $\log Z$ for a lattice—is what makes the search expensive, so each problem carries *surrogates*: cheaper functions of the structure that stand in for it. A surrogate makes a claim, a lower bound, an upper bound, or a point prediction, and the claim is part of the object. An analytic bound is proved (Appendix B) and *certified*: checked against the exact value on every structure an oracle can score, with no violation allowed. A learned predictor is *calibrated* instead, shifted by a quantile of its held-out residuals, and certified to the coverage it claims. The search ranks a neighbourhood by a surrogate and fits only the top candidates; the accepted move is always evaluated in full, so a surrogate changes what is fitted and never what is reported.

For a topology the maximized log-likelihood $\ell^*(\mathcal{T}) = \max_{\boldsymbol{\tau}} \ell(\mathcal{T}, \boldsymbol{\tau})$ is bracketed by two one-pass quantities. Any feasible lengths give a lower bound, and lengths $\hat{\boldsymbol{\tau}}$ solved by non-negative least squares from the pairwise Jukes–Cantor distances $\hat{d}_{ab}$ are feasible:

$$\ell(\mathcal{T}, \hat{\boldsymbol{\tau}}) \;\le\; \ell^*(\mathcal{T}), \qquad \hat{\boldsymbol{\tau}} = \arg\min_{\boldsymbol{\tau} \ge 0} \sum_{a<b} \Big(\sum_{e \in \mathrm{path}(a,b)} \tau_e - \hat{d}_{ab} \Big)^2. \tag{6}$$

Under Jukes–Cantor $\mathbf{P}(t) = a\,\mathbf{I} + (1-a)\,\mathbf{J}/k$ with $a = e^{-kt/(k-1)}$, so a site likelihood is multilinear in the a_e and its maximum over the box $[0,1]^{|E|}$ sits at a vertex, where every branch either forbids a change or forgets the parent's state at cost $1/k$. With the tree rooted at the first leaf, the vertex maximum of site s is $\pi(x_{1,s})\, k^{-F_s(\mathcal{T})}$ for the site's Fitch score F_s, whence

$$\ell^*(\mathcal{T}) \;\le\; \sum_s \log \pi(x_{1,s}) \;-\; F(\mathcal{T}) \log k. \tag{7}$$

Both apply to a subtree as they do to the tree.

For a lattice the partition function is bracketed by two variational quantities. The naive mean-field bound is Gibbs' inequality at a product distribution $q = \prod_i q_i$, tightest at its fixed point:

$$\log Z \;\ge\; \sum_i \sum_\sigma q_i(\sigma)\, h_i(\sigma) + \sum_{\langle i,j \rangle} J_{ij} \sum_\sigma q_i(\sigma)\, q_j(\sigma) - \sum_i \sum_\sigma q_i(\sigma) \log q_i(\sigma). \tag{8}$$

Because $\log Z$ is convex in the parameters, Jensen's inequality over a distribution ρ on spanning trees T, with parameters $\theta(T)$ supported on T and averaging to θ, gives an upper bound each term of which is exact by one leaf-to-root pass:

$$\log Z(\theta) \;\le\; \sum_T \rho(T)\, \log Z\big(\theta(T)\big), \qquad \sum_T \rho(T)\, \theta(T) = \theta, \tag{9}$$

the tree-reweighted bound of Wainwright et al. [40]. From any bracket $L \le \log Z(\beta) \le U$ at inverse temperature β, and $e^{-\beta E_{\min}} \le Z(\beta) \le k^N e^{-\beta E_{\min}}$,

$$-\frac{U}{\beta} \;\le\; E_{\min} \;\le\; \frac{N \log k - L}{\beta}. \tag{10}$$

3 Problem Statement: Phylogenetic Inference

3.1 The model

Character evolution along a branch is a continuous-time Markov chain with rate matrix $\mathbf{Q}$; the transition probabilities over a branch of length t are the matrix exponential $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ [12]. A time-reversible model factors as $\mathbf{Q} = \mathbf{S}\,\mathrm{diag}(\boldsymbol{\pi})$ with $\mathbf{S}$ symmetric, and then satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$.

The simplest such model is Jukes–Cantor on k states: every substitution equally likely and $\boldsymbol{\pi}$ uniform, with $\mathbf{Q}$ scaled so that one unit of branch length is one expected substitution per site,

$$-\sum_{i\in\mathcal{A}} \pi_i \, \mathbf{Q}_{ii} = 1, \tag{11}$$

which puts the transition probabilities in closed form [12, 9]:

$$\mathbf{P}_{ij}(t) = \frac{1}{k} + \Big(\delta_{ij} - \frac{1}{k}\Big)\, \exp\Big(-\frac{k}{k-1}\, t\Big). \tag{12}$$

Rows sum to one, $\mathbf{P}(0) = \mathbf{I}$, and every entry tends to $1/k$ as $t \to \infty$; the general time-reversible model replaces the single rate by $\mathbf{S}$ and $\boldsymbol{\pi}$ and keeps Equation 11.

Given a topology $\mathcal{T}$ over n taxa, branch lengths $\boldsymbol{\tau}$, and an alignment $\mathbf{D}$ of L sites, sites evolve independently and the likelihood marginalizes every assignment z of states to the internal nodes:

$$p(\mathbf{D} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \prod_{s=1}^{L} \sum_{z} \pi_{z_\rho} \prod_{(u\to v)} \mathbf{P}_{z_u z_v}(\tau_v), \tag{13}$$

the product over the branches $(u \to v)$ of the rooted tree with the leaves fixed by the data—k^{n-2} terms per site for a binary tree. Summing it directly is the oracle every faster evaluation of it is checked against.

3.2 As a factor graph

One site is one instance of Equation 1. The variables are the states $x_u \equiv z_u \in \mathcal{A}$ at every node u of the rooted tree, leaves included. The factors are the root prior $\psi_\rho(x_\rho) = \pi_{x_\rho}$, one pairwise factor per branch, $\psi_v(x_u, x_v) = \mathbf{P}_{x_u x_v}(\tau_v)$ for u the parent of v, and one indicator per leaf, $\psi_\ell(x_\ell) = \mathbb{1}[x_\ell = \text{observed}]$, which folds the data in. The bipartite graph is a tree, so Equation 2 is exact, and Z is the site likelihood $p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$; sites being independent (Equation 13), the alignment's likelihood is the product of L such Z. In the Forney form every internal node's state is an edge through an equality factor of degree three—two branches and, at the root, the prior—which is the drawing Section 6 uses.

3.3 The algorithm: simulation

The generative direction of Equation 13 is a pre-order walk: draw the root state, then each node's state from the row of its parent's state. One site is Algorithm 1; L sites are L independent runs, and the empirical substitution frequencies along any branch converge to Equation 12, which is the check a simulator is held to—against the closed form, and never against the likelihood that will later be fitted to its output.

Algorithm 1 Forward simulation of one site under $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$

draw $z_\rho \sim \boldsymbol{\pi}$
for each non-root node v in pre-order, with parent u **do**
    draw $z_v \sim \mathbf{P}_{z_u,\cdot}(\tau_v)$
end for
return the leaf states $(z_\ell)_{\ell \in \text{leaves}(\mathcal{T})}$

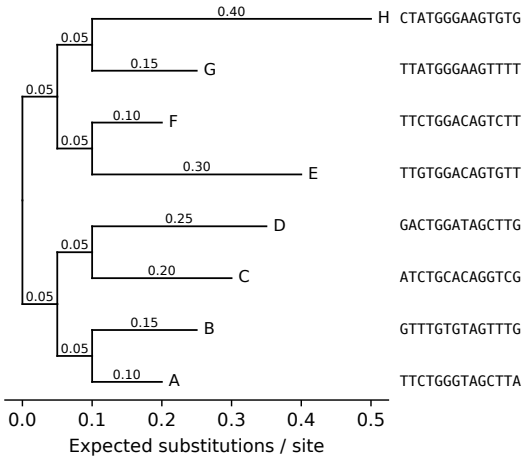

Figure 1: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site. The first 14 simulated sites are shown beside each leaf, in tree order, so the alignment the likelihood consumes can be read off against the topology that generated it.

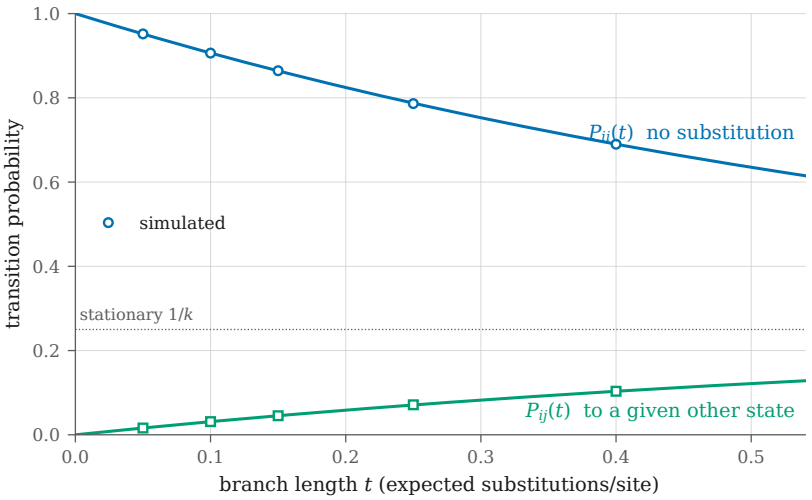

Figure 2: Closed-form Jukes-Cantor transition probabilities for k = 4 (curves) against the frequencies the simulator produced (markers), one pair per branch of the 5-branch fixture tree. Simulated at seed 20260902 over 200000 sites. Largest deviation 1.07e-03, within the fixture's declared Monte Carlo tolerance of 1e-02. The simulator is validated against this analytic form, never against the likelihood code it feeds.

3.4 The algorithm: pruning

Felsenstein's pruning evaluates that marginal in $O(nLk^2)$ rather than $O(Lk^{n-2})$, by a post-order recursion carrying a partial likelihood $L_u(i)$—the probability of the data below node u given that u is in state i:

$$L_u(i) = \prod_{v \in \text{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(t_v)\, L_v(j), \tag{14}$$

with $L_u(i) = \mathbb{1}[\,u \text{ observed in state } i\,]$ at a leaf, and the site likelihood read off at the root,

$$p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \sum_{i \in \mathcal{A}} \pi_i\, L_\rho(i) \tag{15}$$

[12, 9]. Products of many small numbers underflow, so each node's partials are rescaled and the logarithm of the scaling accumulated separately; the rescaling must remain differentiable, since $\boldsymbol{\tau}$ and $\mathbf{Q}$ are what the continuous optimization fits.

Equation 14 is Equation 2 run leaf to root with the root as the last variable: $L_u(i)$ is the product over u's children of the factor-to-variable messages $m_{\psi_v \to u}(i)$, each of which sums the branch factor against the message arriving from below, and Equation 15 is the final factor-to-variable message from the prior. Replacing the sum by a maximum gives the most probable joint assignment of ancestral states; in the limit of short branches its cost counts substitutions, which is Fitch parsimony [12], the second criterion the search is scored against.

Regime. Exact on every input, because a tree has no cycle. The one failure mode is numerical—underflow of a product of n factors each below one—and it is removed by the rescaling, not tolerated.

What pins it. Direct marginalization, Equation 13, at the sizes it allows, to a relative tolerance (Section 8.5). Two invariants past those sizes: the pulley principle, that a reversible model's likelihood does not depend on where the root is placed along a branch, and a gradient against central differences relative to its own norm. And the recovery test of Section 8: on simulated data the fitted branch lengths cover the truth at their nominal rate, and the search recovers the generating topology at a stated site count.

3.5 The algorithm: discrete search

The number of unrooted binary topologies on n taxa is $(2n-5)!!$ [36], so exhaustive enumeration is an oracle at small n and nothing more. Two neighbourhoods are standard. *Nearest-neighbour interchange* (NNI) swaps the subtrees adjacent to one internal edge, giving $|\mathcal{N}_{\text{NNI}}(\mathcal{T})| = 2(n-3)$. *Subtree prune and regraft* (SPR) detaches a subtree and reattaches it at another edge, giving $2(n-3)(2n-7)$ and containing NNI. Neither is complete in one step; both are complete in the transitive closure, which for NNI is the connectivity of the associated graph [12] and is inherited by SPR because it dominates NNI.

Hill climbing over either neighbourhood fits the continuous parameters of every candidate and moves to the best; its budget is counted in candidates scored, never in seconds, so a run reproduces from its seed. A candidate that shares all but a few branches with its parent may start its fit from the parent's lengths, a branch being identified by the leaf bipartition below it rather than by a node's name; and a neighbourhood may be *ranked* by one evaluation at those lengths before the few best are fitted, the accepted move always in full, so every reported score is an optimum. Whether the ranking finds the fitted best is a property of the move set and is measured per move set, not assumed.

What pins it. Enumeration of the $(2n-5)!!$ topologies referees the search below eight taxa: the climb must reach the enumerated maximum, and the neighbourhood counts above are asserted exactly over every topology. A warm start is pinned by reaching the cold fit's optimum; a cached partial likelihood by equalling the recomputed one bitwise, since it is the same arithmetic in the same order.

4 Problem Statement: Potts Models in an External Field

4.1 The model

On a graph $\mathcal{G}$, each node i carries a spin $\sigma_i \in \{1, \ldots, q\}$ and the energy is

$$H(\sigma) = -\sum_{\langle i,j \rangle} J_{ij}\, \delta(\sigma_i, \sigma_j) - \sum_i h_i(\sigma_i), \tag{16}$$

with couplings J_{ij} and an external field h_i [27]. The Boltzmann distribution is $p(\sigma) \propto \exp(-H(\sigma))$, and its normalizer sums over $q^{|V|}$ configurations.

4.2 As a factor graph

The variables are the spins, $x_i \equiv \sigma_i$. The factors are one unary $\psi_i(\sigma_i) = e^{h_i(\sigma_i)}$ per node and one pairwise $\psi_{ij}(\sigma_i, \sigma_j) = e^{J_{ij}\,\delta(\sigma_i,\sigma_j)}$ per edge, so Equation 1 is the Boltzmann distribution and Z its normalizer. At a temperature T the factors are raised to the power $\beta = 1/T$, which is the same graph with J and h scaled; nothing else about the model changes, and that is why a tempered chain can be held to the unscaled model's enumeration. The graph is a tree exactly when $\mathcal{G}$ is, which is the regime every exact claim below is confined to.

4.3 Why an odd cycle is the whole story

A two-state antiferromagnet ($J < 0$) wants every edge to disagree, which is possible exactly when $\mathcal{G}$ is 2-colourable. So every bipartite lattice—a chain, a square lattice—has a trivial, unfrustrated ground state and says nothing about a hard problem. A geometry containing odd cycles is what makes the ground state non-trivial, and where a counting argument closes exactly it fixes that ground state at every size. This is the difference between a fixture that exercises an optimizer and one that does not.

4.4 The algorithm: belief propagation

Exact marginals come from enumeration at small sizes and from belief propagation on a tree, where it is exact; on a loopy graph it is an approximation whose accuracy is a property of the graph rather than of the implementation [22, 14]. It is Equation 2 with every factor pairwise or unary: composing the variable-to-factor message with the unary factor gives the node-to-node form, in which the message from i to a neighbour j is

$$m_{i\to j}(\sigma_j) \;\propto\; \sum_{\sigma_i} \exp\big(J_{ij}\,\delta(\sigma_i,\sigma_j) + h_i(\sigma_i)\big) \prod_{l \in \partial i \setminus j} m_{l\to i}(\sigma_i), \tag{17}$$

iterated to a fixed point, with beliefs $b_i(\sigma_i) \propto e^{h_i(\sigma_i)} \prod_{l\in\partial i} m_{l\to i}(\sigma_i)$ at a node and likewise on an edge. The messages are carried in the logarithm and normalized every sweep, because a coupling of a few units on a small lattice already puts a product of exponentials past what a linear recursion holds. The Bethe free energy of the fixed point,

$$F_{\mathrm{Bethe}} = \sum_{\langle i,j \rangle} \sum_{\sigma_i,\sigma_j} b_{ij} \left(E_{ij} + \log b_{ij}\right) + \sum_i \sum_{\sigma_i} b_i\, E_i - \sum_i (d_i - 1) \sum_{\sigma_i} b_i \log b_i, \tag{18}$$

with $E_{ij} = -J_{ij}\,\delta(\sigma_i, \sigma_j)$, $E_i = -h_i(\sigma_i)$ and d_i the degree of i, equals $-\log Z$ exactly on a tree and approximates it on a loopy graph [42, 27]; Appendix A.1 states why. A fixed point the iteration never reached is not an estimate of anything, so non-convergence is a refusal rather than a number.

Regime. Exact on a tree, in two passes. On a loopy graph the flooding schedule is iterated with damping to a tolerance and returns the Bethe approximation, and the tree schedule is refused rather than run, because a number that looks exact and is not is the worst output an evaluator can produce.

What pins it. On a tree, $\log Z$ and every marginal against enumeration to a stated relative tolerance. On a loopy graph nothing asserts agreement with the truth—that would assert something false—and what is asserted is the structure the physics requires: exact at zero coupling, and a deviation from the exact transfer matrix that grows away from it and is reported as a measurement.

4.5 The algorithm: ground states as cuts

With two states write $\sigma_i \in \{-1, +1\}$. The pairwise term of Equation 16 is constant on agreeing edges and pays J_{ij} on disagreeing ones, so minimizing H minimizes the weight of the *cut*—the set of edges whose endpoints differ—plus the field:

$$H(\sigma) = \sum_{\langle i,j\rangle : \sigma_i \neq \sigma_j} J_{ij} - \sum_i h_i(\sigma_i) + \text{const.} \tag{19}$$

When every $J_{ij} \geq 0$ the pairwise term is submodular, the field becomes a terminal edge to a source or a sink, and the ground state is a minimum s–t cut, exact by max-flow in polynomial time [19, 8]. When couplings are negative the problem is maximum cut, which is NP-hard; the semidefinite relaxation rounded by a random hyperplane gives a cut whose expected weight is at least $\alpha_{\mathrm{GW}} = 0.87856$ of the optimum,

$$\mathbb{E}[\,w(\text{cut})\,] \;\geq\; \alpha_{\mathrm{GW}}\, \max_{\sigma} w(\sigma), \qquad \alpha_{\mathrm{GW}} = \min_{0<\vartheta\leq\pi} \frac{2}{\pi}\,\frac{\vartheta}{1-\cos\vartheta}, \tag{20}$$

and the relaxation's own value bounds the optimum from above [17]. The bound is the claim that holds at every size; the realized ratio on an instance is measured beside it and is never quoted as the guarantee.

Regime. Exact where the couplings are submodular and refused rather than approximated where they are not (Section 8): a wrong ground state on a structured instance is indistinguishable from a right one at any size worth solving. The relaxation runs on any instance and certifies a ratio, not an optimum.

What pins it. Enumeration over $2^{|V|}$ configurations where it reaches; where it does not, a *reduction*: on a bipartite graph the maximum cut is every edge, on a uniform ferromagnet the ground state is constant, and a construction error in the cut graph yields a labelling that is merely worse, which only a regime where the general construction must reproduce a simpler validated one will catch. A fixture whose answer is trivial measures nothing, so the odd cycles of Section 4 are what the instances carry.

4.6 The algorithm: alpha expansion

With $q > 2$ states the ground state is NP-hard in general, and alpha expansion approximates it by a sequence of exact two-state problems [5]. An *expansion* on label α lets every site keep its label or switch to α; the best such move is the minimum cut of a two-state graph built from the current labelling, exact when the pairwise term is a metric on labels, which $\delta(\sigma_i, \sigma_j)$ is. Cycling over α until no expansion lowers the energy reaches a local minimum within a constant of the global one,

$$H(\hat{\sigma}) \;\le\; 2\,c\,H(\sigma^\star), \qquad c = \max_{\alpha\neq\beta'} d(\alpha,\beta') \big/ \min_{\alpha\neq\beta'} d(\alpha,\beta'), \tag{21}$$

so a factor of two for the Potts metric, where $c = 1$. The guarantee assumes each expansion is solved to optimality; where it is solved approximately the certificate is optimistic, and what can be asserted is the symptom—a value an exact solve could not have produced.

What pins it. The two-label case must reproduce the exact minimum cut of Section 4.5 exactly; enumeration referees the multi-label case where it reaches; and the realized ratio against the enumerated optimum is measured, since the bound is a theorem and the ratio is a number.

4.7 The algorithm: sampling

The single-site heat bath draws each spin from its conditional,

$$p(\sigma_i \mid \sigma_{\partial i}) \;\propto\; \exp\Big(\beta\big[\textstyle\sum_{j\in\partial i} J_{ij}\,\delta(\sigma_i,\sigma_j) + h_i(\sigma_i)\big]\Big), \tag{22}$$

one site at a time; a *sweep* is one update of every site. Near a critical coupling correlated regions span the lattice and single-site moves decorrelate slowly. Cluster moves flip such regions together [39, 41, 28]: in the Edwards–Sokal representation [11] a bond is placed between equal neighbours with probability $1 - e^{-\beta J_{ij}}$, and each bond-connected cluster is recoloured uniformly—every cluster at once in Swendsen–Wang, one cluster grown from a random seed in Wolff. The bond construction is exact at zero field only. In a field the recolouring is accepted with the Metropolis probability on the field energy alone,

$$a = \min\Big\{1,\; \exp\Big(\beta \textstyle\sum_{i\in\mathcal{C}} \big[h_i(\nu) - h_i(\mu)\big]\Big)\Big\}, \tag{23}$$

for a cluster $\mathcal{C}$ recoloured from μ to ν, which restores detailed balance (Appendix A.2).

Regime. Every move preserves the Boltzmann distribution at β, so a sweep may not stop on a state-dependent condition: composing a *number* of steps chosen from the outcome biases toward the states that triggered the stop. The bond probability is a probability only for $J_{ij} \ge 0$, and an antiferromagnet has no like-spin clusters to flip, so the cluster moves refuse a negative coupling rather than run badly.

What pins it. A sampler is validated by the distribution it converges to and never by inspection: at an enumerable size the exact Boltzmann distribution is available, and a chain is held to it by a goodness-of-fit test at a declared significance, thinned by several autocorrelation times because successive sweeps are not independent draws. The test's power is itself pinned—a move set with the field accept step removed runs, mixes, and is rejected.

4.8 Temperature, annealing and tempering

A temperature T scales the model, H/T, so a chain at T targets $\exp(-H/T)$; a *schedule* $T(t)$ over a budget of t steps is one object, shared by every consumer of it, with both endpoints reached exactly at the declared steps and a step past the end refused. Simulated annealing is the

sampler on a decreasing schedule, returning the lowest-energy state seen [21]; at $T \to 0$ the heat bath is the argmin over each site's conditional, which is iterated conditional modes, so annealing and greedy descent are one search separated by the schedule alone.

Parallel tempering runs R replicas at fixed temperatures $T_1 < \cdots < T_R$ and, after each sweep, proposes to exchange the configurations of adjacent replicas, accepting with

$$a_{ij} = \min\big\{1,\ \exp\big[(\beta_i - \beta_j)(E_i - E_j)\big]\big\}, \tag{24}$$

the ratio of the joint target, a product of tempered marginals [38, 16, 10]; Appendix A.3 derives it. The hot replicas cross barriers the cold one cannot, and an exchange carries what they find down the ladder. Each replica draws from its own generator, spawned from one parent that then draws only the exchange uniforms, so no two replicas share a stream and one seed reproduces the run.

What pins it. Every replica's marginal must still be $\exp(-H/T_r)$, enumerated from the *unscaled* model, with exchanges on: a wrong exchange ratio contaminates the cold replica with hot configurations and only that test says so. Its power is pinned by the paired negative—an exchange that omits the energy term still runs and still mixes, and is rejected on every replica. Whether tempering or annealing beats restarts of descent at an equal budget of sweeps is not a property of the algorithms but of the instance, and is measured on the frustrated, past-enumeration instances the roadmap needs, never asserted in general.

5 Problem Statement: Hidden Markov Models

5.1 The model

A hidden path $\mathbf{Z} = (z_1, \ldots, z_T)$ over k states evolves by an initial distribution $\boldsymbol{\pi}$ and a transition matrix $\mathbf{A}$; each position emits an observation from an emission family $\mathcal{E}$ indexed by its state. The evidence marginalizes k^T paths.

5.2 As a factor graph

The variables are the hidden states, $x_t \equiv z_t$. The factors are $\psi_1(z_1) = \pi_{z_1}\,\mathcal{E}_{z_1}(y_1)$ at the first position and, for $t > 1$, one pairwise $\psi_t(z_{t-1}, z_t) = \mathbf{A}_{z_{t-1} z_t}$ per transition and one unary $\mathcal{E}_{z_t}(y_t)$ per emission: the observation y_t is folded into the table, so a probability and a density enter the same way and the graph is over latent variables only. The chain is a tree, and Z is the evidence $p(\mathbf{D})$. An interior state sits on three factors, so in the Forney form it passes through an equality node—the drawing Section 6 uses for each class's chain.

The emission family is the only part that knows what an observation is. Everything else—the recursions, the expectation step, path enumeration—needs three things from it: draw an observation given a state, score one, and re-estimate itself from posterior weights. Separating it is what lets one set of recursions serve a finite alphabet, a real observation, and a count.

A density is not a probability. For a family over a countable alphabet the evidence is a probability, so $\log p(\mathbf{D}) \le 0$. For a family over the reals it is a *density* and carries no such bound. A check written against the first is false under the second, so what may be asserted is keyed on the support the model declares.

5.3 The algorithm: forward–backward

The forward recursion is Equation 14 on a chain:

$$\alpha_t(i) = \Big[\sum_j \alpha_{t-1}(j)\,\mathbf{A}_{ji}\Big]\,\mathcal{E}_i(y_t), \qquad p(\mathbf{D}) = \sum_i \alpha_T(i). \tag{25}$$

Pruning on a caterpillar tree carrying one observed leaf per internal node *is* this recursion [9, 22]: the three problem classes are one marginalization over three structures, not three unrelated models sharing an optimizer. In the terms of Equation 2, α_t is the message arriving at z_t from the left multiplied by the emission factor, the backward recursion β_t is the same pass from the right, and the posterior at a position is their normalized product,

$$p(z_t = i \mid \mathbf{D}) = \frac{\alpha_t(i)\,\beta_t(i)}{p(\mathbf{D})}, \qquad \beta_t(i) = \sum_j \mathbf{A}_{ij}\,\mathcal{E}_j(y_{t+1})\,\beta_{t+1}(j), \tag{26}$$

which is the belief b_t of Section 1. Max-product on the same chain is Viterbi,

$$\delta_t(i) = \Big[\max_j\ \delta_{t-1}(j)\,\mathbf{A}_{ji}\Big]\,\mathcal{E}_i(y_t), \tag{27}$$

with the path read back from the argmax at each step; on a chain with a unique maximum the max-marginals' argmax at every position is that path [9].

Regime. Exact for every length, the chain being a tree. The one hazard is the same underflow pruning meets, and the same remedy applies: the recursion runs in the logarithm.

What pins it. The sum over every one of the k^T paths, written out term by term with no recursion, referees the evidence, the posteriors and the Viterbi path at the lengths it allows; a gradient against central differences past them. The enumeration is the oracle two decoders are separated by, so it cannot be validated by a decoder, and it is pinned in turn against algebra: quantities that can be worked out by hand on a two-state, two-step chain.

5.4 The algorithm: expectation-maximization

Baum–Welch alternates a posterior over hidden states given the parameters with a re-estimate of the parameters given that posterior, and increases the likelihood at every iteration [25]. The initial distribution and the transitions re-estimate in closed form for every family; the emission re-estimate is the family's own, and *whether it is a formula* is the property that separates the families. A weighted mean is; a dispersion parameter whose score involves the digamma function is not, and its M step is itself an optimization—which is the case that says whether an interface built around closed forms was built correctly.

What pins it. Monotonicity, asserted at every iterate; agreement of the fixed point with a gradient fit of the same likelihood; and recovery of simulated parameters up to the permutation of hidden states, with the alignment part of the test (Section 8). Where the family's M step is itself an optimization, its refusals at the boundary of the parameter space are pinned too: a dispersion the data cannot identify is refused, not clamped.

5.5 Two decodings, and they are different answers

Viterbi returns the single path of highest joint probability; posterior decoding returns, at each position independently, the state of highest marginal probability. Where they disagree, the posterior-decoded sequence can be a path the model assigns zero probability to, because nothing

constrains consecutive choices to be compatible. They agree on almost every instance, which is exactly why an implementation that computes one and reports the other survives a test suite with no case where they diverge. Equation 27 and the argmax of Equation 26 are those two decodings; the fixture on which they differ is constructed and kept, because it is the only kind of case that can tell an implementation of one from an implementation of the other.

6 Problem Statement: The Coupled Spatio-Sequential Model

6.1 The model

Observations x_{sn} are indexed by a spatial node n of a graph $\mathcal{G}$ and a sequential position $s = 1, \dots, S$. Each node carries a class label $\ell_n \in \{1, \dots, M\}$; each class m carries its own hidden chain $k_{s,m} \in \{1, \dots, K\}$; and the observation at (s, n) is emitted by the chain of the class the node belongs to, under that class's emission parameters θ_m. The joint is Equation 3. Its three terms are the three problem classes above: a Potts prior on the labels,

$$\log p(\ell) = \beta \sum_{\langle n,n' \rangle} J_{nn'}\, \delta(\ell_n, \ell_{n'}) - \log Z_{\text{Potts}}, \qquad J_{nn'} \geq 0, \tag{28}$$

ferromagnetic so that neighbouring nodes prefer one class—the penalty on disagreement, $-\beta \sum J_{nn'} \mathbb{1}[\ell_n \neq \ell_{n'}]$, is the same distribution up to a constant; one hidden Markov chain per class, in the simplest case with a circulant transition,

$$\mathbf{A}_m(i, j) = t\, \delta_{ij} + \frac{1 - t}{K - 1}\,(1 - \delta_{ij}), \tag{29}$$

whose self-transition rate t and initial distribution Π_m are fitted; and a *gated* emission, $\mathbb{1}[\ell_n = m] \log p(x_{sn} \mid k_{s,m}, \theta_m)$, which is what couples the two. Marginalizing k recovers a Potts model in a field the chains define; marginalizing ℓ recovers a mixture of hidden Markov models. Neither marginal is tractable on its own, and the structure of the joint is what dictates the estimator.

6.2 As a factor graph

The variables are the N labels and the SM chain states. The factors are the pairwise Potts factors of Section 4 on the labels, the chain factors of Section 5 on each class's states, and one gated emission factor per (n, s, m) over the pair $(\ell_n, k_{s,m})$, equal to $p(x_{sn} \mid k_{s,m}, \theta_m)$ when $\ell_n = m$ and to one otherwise. A label sits on every emission factor of its node and on every Potts factor of its edges; in the Forney form it passes through an equality node of that degree, and the gated factors are the higher-degree nodes that join the lattice to the chains (Figure 3). The graph has cycles, so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumeration on an instance small enough to enumerate—a 2×2 lattice with $M = 2$, $K = 2$, $S = 3$ has $2^4 \cdot 2^{12}$ joint states.

6.3 The estimator: block-coordinate ascent

The blocks are the emission parameters θ and the labels ℓ, with the chain states marginalized. For θ given ℓ, the gradient of the marginal likelihood is the posterior-weighted gradient of the emission terms,

$$\nabla_\theta \log p(x \mid \ell, \theta) = \sum_{n,m} \mathbb{1}[\ell_n = m] \sum_{s, k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta, \ell; x)\, \nabla_\theta \log p(x_{sn} \mid k_{s,m}, \theta_m), \tag{30}$$

where $\mathbb{Q}$ is the per-class posterior of Equation 26: this is expectation–maximization with the E step forward–backward on each class's chain over the nodes assigned to it, and t and Π_m

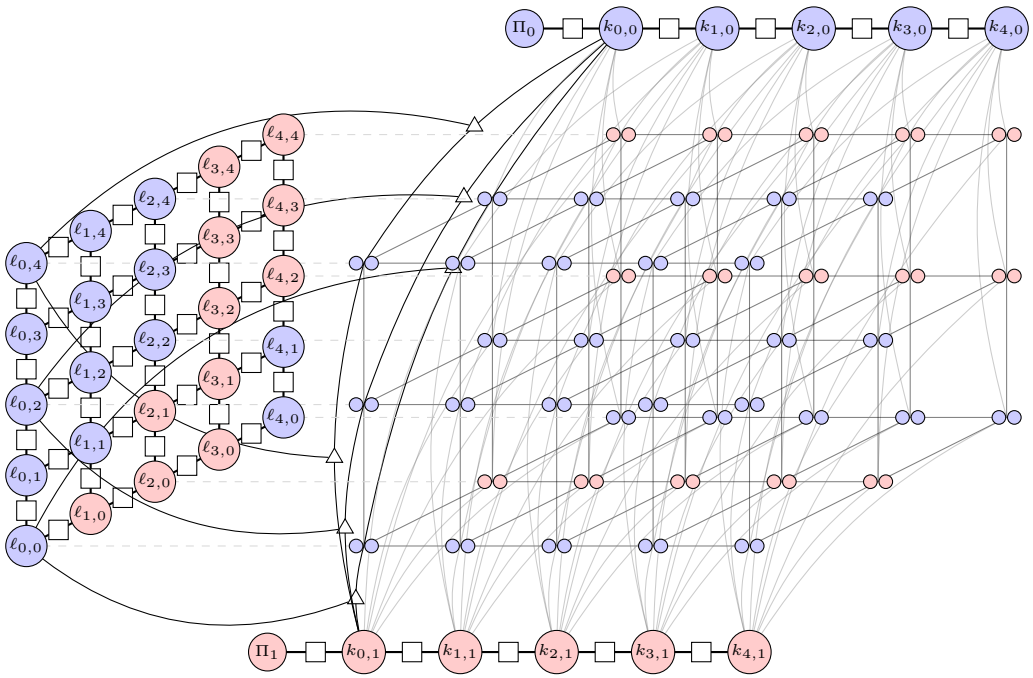

Figure 3: The coupled spatio-sequential model as a Forney-style factor graph. Each spatial node n carries a class label $\ell_n \in \{1, \ldots, M\}$ (circles on the lattice, coloured by class for $M = 2$); each class m carries a hidden chain $k_{s,m} \in \{1, \ldots, K\}$ with initial distribution Π_m (the two chains at the sides). Pairwise factors (squares) carry the Potts prior on the labels, Equation 28, and the chain prior, Equation 29. The gated emission factors (triangles) join one label to one chain state and contribute $\log p(x_{sn} \mid k_{s,m}, \theta_m)$ only when $\ell_n = m$; the observations x_{sn} are the small circles along each column. Every second observation column is drawn along each spatial axis, and an illustrative subset of the gated factors, so the structure is legible.

re-estimate in closed form. For ℓ given θ, the same posterior defines a per-node, per-class external field,

$$H_{nm}(\theta, \theta'; \ell) \equiv - \sum_{s, k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta', \ell; x) \, \log p(x_{sn} \mid k_{s,m}, \theta), \tag{31}$$

and the label update is the ground state of a Potts model in that field,

$$\ell^\star = \arg\min_{\ell} \; \beta \Big[\sum_{\langle n, n' \rangle} J_{nn'} \mathbb{1}[\ell_n \neq \ell_{n'}] + \sum_{n,m} \mathbb{1}[\ell_n = m] \, H_{nm} \Big], \tag{32}$$

which is the problem of Section 4.6, solved by alpha expansion with its bound, descended by iterated conditional modes, or sampled by an annealed cluster move with a field (Section 4.7). Each block does not decrease the joint likelihood, which is the monotonicity every expectation–maximization here is held to.

Why a cluster move. Under an effective spatial prior each class spans a contiguous region, and a single-site move must cross a region one node at a time against the prior's penalty; iterated conditional modes freezes into spurious disjoint clusters, and single-site annealing slows critically for the same reason. The Wolff move of Section 4.7 recolours a whole cluster at once, and Algorithm 2 is that move with the field of Equation 31 in its accept step. At $\beta \to 0$ it is a single-site Gibbs update; at $\beta \gg 1$ it is the greedy entrapment of iterated conditional modes; annealing in β moves between them.

Algorithm 2 One Wolff update on the labels, in the external field

draw a root ρ uniformly from the nodes; $\mu \leftarrow \ell_\rho$; $\mathcal{C} \leftarrow \{\rho\}$; queue $Q \leftarrow \{\rho\}$
while $Q \neq \varnothing$ **do**
    pop n from Q
    for each neighbour n' of n with $\ell_{n'} = \mu$ and $n' \notin \mathcal{C}$ **do**
        with probability $1 - e^{-\beta J_{nn'}}$: add n' to $\mathcal{C}$ and to Q
    end for
end while
draw a new label ν uniformly from $\{1, \ldots, M\}$
$\Delta H \leftarrow \sum_{n \in \mathcal{C}} (H_{n\nu} - H_{n\mu})$
with probability $\min\{1, e^{-\beta \Delta H}\}$: $\ell_n \leftarrow \nu$ for every $n \in \mathcal{C}$

6.4 The initializer: data-sampled burn-in

A block ascent on this joint is only as good as where it starts, and a start that ignores the graph and the emission family—k-means++ on the raw observations, say—leaves degenerate classes and false positives that the ascent then polishes. Algorithm 3 anneals in β from zero, so that early on the labels are nearly free and the chains and emission parameters are fitted to what the data alone supports, and the spatial prior is tightened only as the classes separate. Its seeding step, EmissionMixture++, is k-means++ with the emission family's own negative log-density in place of squared Euclidean distance [2]: candidates are drawn proportionally to how badly the current parameters explain them, constrained to the observations the current labels and states assign to the class being seeded. The $8(\ln K + 2)$ expected-cost bound of k-means++ is a statement about squared distance and is not inherited; what is measured instead is recovery against a uniform start at an equal budget, over seeds, reported whether it wins or not.

What pins it, and what is built. The structure and its evaluator are built: the graph of Equation 3 is constructed from the three adapters and its log-density equals the joint written

Algorithm 3 Data-sampled burn-in on the coupled graph

$\beta \leftarrow 0$; $H \leftarrow 0$; $\ell_n \sim \mathrm{Uniform}\{1, \dots, M\}$ for every node
$\theta \leftarrow$ EmissionMixture++(β, K, θ, x)
while $\beta < \beta_0$ **do**
    WolffUpdate(β, ℓ, J, H) ▷ Algorithm 2
    for each class m **do** draw the chain $k_{\cdot,m}$ by block Gibbs given ℓ, θ
    end for
    for each state k **do** $\theta_k \leftarrow$ EmissionMixture++$(\beta, K, \theta, x_k, \ell)$ ▷ state-constrained
    end for
    $\mathbb{Q} \leftarrow$ forward–backward per class (Equation 26); M step for θ, t, Π
    $H_{nm} \leftarrow$ Equation 31
    $\beta \leftarrow \beta + \Delta\beta$
end while
return ℓ, θ

out on every assignment of an enumerable instance. The fixture with its enumeration oracle, the E step and the field, the block ascent with the three label solvers, and the initializer are the remaining parts, each with its own pin: label and state marginals of the simulator against enumeration; the gradient identity Equation 30 against central differences; the joint non-decreasing across every block; label accuracy up to permutation against the planted labels at an equal budget of sweeps, for each of the three label solvers, so the claim that a cluster move escapes what iterated conditional modes freezes into is a measurement here and not a premise.

7 Problem Statement: LDPC Decoding

7.1 The model

A binary linear code of length n is the null space of a parity-check matrix $\mathbf{H} \in \{0,1\}^{m \times n}$ over GF(2): the codewords are the c with $\mathbf{H}c = 0$, there are 2^k of them with $k = n - \operatorname{rank} \mathbf{H}$, and the rate is k/n. A low-density code makes $\mathbf{H}$ sparse. Gallager's regular (d_v, d_c) ensemble [15] puts d_v ones in every column and d_c in every row: $\mathbf{H}$ is d_v bands of n/d_c rows, the first band's row i covering bits id_c to $(i+1)d_c - 1$ and every other band the first under a uniformly random permutation of the columns, so n is a multiple of d_c and $m = nd_v/d_c$. The rows of one band sum to the all-ones vector, so at least $d_v - 1$ rows are dependent and $k \geq n - m + d_v - 1$. The instance this repository carries is $(3, 6)$ at $n = 19{,}998$, the multiple of six nearest the 20,000 the ticket names: $m = 9{,}999$ checks, 59,994 nonzeros, design rate 1/2.

A codeword is sent through a memoryless binary-input channel and y is received. Three channels are supported, each seen only through its log-likelihood ratio,

$$L_i = \log \frac{p(y_i \mid c_i = 0)}{p(y_i \mid c_i = 1)} = \begin{cases} (1 - 2y_i) \log \dfrac{1-p}{p} & \text{binary symmetric, flip probability } p \\ 0 \text{ if erased, else } \pm\infty \text{ with the sign of } 1 - 2y_i & \text{binary erasure, erasure probability} \\ 2y_i/\sigma^2 & \text{binary-input Gaussian, } y_i = (1 - 2c \end{cases} \tag{33}$$

positive when the received symbol favours zero. The posterior over the code is then

$$p(c \mid y) \;\propto\; \mathbb{1}[\mathbf{H}c = 0] \, \exp(-c \cdot L), \tag{34}$$

since $\log p(y \mid c) = \sum_i \log p(y_i \mid 0) - \sum_i c_i L_i$ and the first sum is the same for every word. Decoding asks two different questions of Equation 34: the *bitwise* MAP, each bit at the mode of its own marginal, which minimizes the bit error rate and need not be a codeword; and the

blockwise MAP, the single most probable codeword, which minimizes the block error rate. They are different answers, as Section 5 says of the chain, and the fixture that separates them is a test.

7.2 As a factor graph

The variables are the bits, $x_i \equiv c_i \in \{0,1\}$. The factors are a unary $\psi_i(c_i) = e^{-c_i L_i}$ per bit, the channel folded in as an observation is everywhere in this document, and a parity factor $\psi_a(c_{\partial a}) = \mathbb{1}[\bigoplus_{i\in\partial a} c_i = 0]$ per row of $\mathbf{H}$, a hard constraint of degree d_c whose log table is zero on even-parity assignments and $-\infty$ elsewhere. This is the Tanner graph of the code; a bit sits on d_v parity factors and one unary, so in the Forney form every bit passes through an equality node of degree $d_v + 1$. The graph has cycles—a random draw of the ensemble does not avoid short ones—so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumerating the 2^k codewords where $2^k \le 200{,}000$ (a $(3,6)$ code at $n = 24$ has $k \ge 14$) and from the tree schedule on a cycle-free code, where message passing is exact.

7.3 The algorithm: sum-product in the log domain

A binary variable's message is one number, its log-odds, and Equation 2 on a parity factor has a closed form [15, 25, 35]. From bit i to check a, and from check a to bit i,

$$m_{i\to a} = L_i + \sum_{b\in\partial i\setminus a} m_{b\to i}, \qquad m_{a\to i} = 2\operatorname{artanh} \prod_{j\in\partial a\setminus i} \tanh\frac{m_{j\to a}}{2}, \qquad \tilde{L}_i = L_i + \sum_{b\in\partial i} m_{b\to i}, \tag{35}$$

where the check update follows from $\tanh(L/2) = p(0) - p(1)$ and the fact that the parity of independent bits has a difference of probabilities equal to the product of theirs. The hard decision $\hat{c}_i = \mathbb{1}[\tilde{L}_i < 0]$ is the bitwise MAP under the Bethe beliefs. Replacing the sum by a maximum in Equation 2 gives max-product, whose log-domain check update is

$$m_{a\to i} = \Big(\prod_{j\in\partial a\setminus i} \operatorname{sgn} m_{j\to a}\Big) \min_{j\in\partial a\setminus i} |m_{j\to a}|, \tag{36}$$

the min-sum decoder, whose decision is the blockwise MAP and is exact on a cycle-free code with a unique maximum; on a loopy code it is the cheaper and the weaker of the two, and the gap is measured rather than assumed.

Three details are load-bearing in the implementation. The schedule is flooding: every check reads the bit messages of the previous half-iteration and every bit reads the check messages just written, one Jacobi sweep per direction over all edges at once, which is the schedule the general implementation runs and so the one it is compared against. The loop stops at the first iteration whose hard decision has every bit decided and satisfies $\mathbf{H}\hat{c} = 0$, the syndrome check, and otherwise at an iteration cap, which is a decoding failure and the event a block error rate counts. And messages are clipped at $\pm L_{\max}$ with $L_{\max} = 30$ before the check update, because a certain bit's ratio is infinite on the erasure channel and the leave-one-out sums in Equation 35 are formed by subtraction: $\tanh(L_{\max}/2)$ is below one by 1.9×10^{-13} in double precision, and a belief formed under the cap differs from the exact one by less than $e^{-L_{\max}}$ in probability, which the oracle tests absorb.

7.4 The all-zero codeword

Every simulation past the reach of an encoder sends the zero word, and the argument that this loses nothing is standard [35, Sec. 4.1]. Each channel of Equation 33 is output symmetric: sending $c_i = 1$ instead of 0 negates L_i in distribution, and on the symmetric and erasure channels it negates it on every realization, since a flip or an erasure is applied to the bit whatever its value. Both check updates are odd in each argument and the bit update is linear, so negating the

ratios at the bits where $c_i = 1$ negates every message and every posterior at exactly those bits, and the hard decision under c is the hard decision under 0 with c added: the error pattern $c \oplus \hat{c}$ has the same distribution whatever codeword was sent. Bit and block error rates measured on the zero word are therefore the rates on any codeword. Where an encoder exists—Gauss–Jordan elimination over GF(2) to a systematic form, at $n \leq 512$—the identity is checked per realization on a non-trivial codeword, so the shortcut is itself pinned.

What pins it. Four oracles, none sharing code with the decoder. On a cycle-free code the tree schedule of the general implementation and the enumeration over all 2^k codewords are exact, and the decoder equals both; min-sum equals the general max-product there and returns the enumerated maximum-likelihood codeword, with the margin over the runner-up pinned so that no tie is being broken. On loopy codes the decoder and the general damped flooding are the same Bethe approximation and reach the same fixed point, which is agreement and not truth. The encoder is pinned by $\mathbf{H}c = 0$ and the offsets layout by the dense product it stands in for. And at size, where no enumeration reaches, the closed form is density evolution (Appendix A.4): on the erasure channel the $(3, 6)$ ensemble has a threshold $\epsilon^\star = 0.4294$ below which the infinite-length decoder resolves every erasure and above which it does not, and the 19,998-bit code is held to bracketing it.

8 The Properties That Referee Each Method

An algorithm is accepted here against a property that is true independently of this implementation. The properties are not interchangeable, and which one applies is a statement about the method rather than a matter of convenience. Each *What pins it* paragraph above names one of the five kinds below, and the regression suite marks every check with the kind it is: an oracle, a simulated truth, a mathematical invariant, an edge case, or a structural property of the code. A check that fits none of them is either a missing category or a test with nothing to assert.

8.1 Exact answers, where an oracle exists

Where a quantity can be computed a second way that shares no recursion with the first, equality is asserted to a stated tolerance. Direct marginalization over k^{n-2} internal assignments referees pruning; a sum over k^T paths referees the forward recursion; exhaustive enumeration over $q^{|V|}$ configurations referees a Potts marginal; enumeration of $(2n-5)!!$ topologies referees a search. Every one is exponential, and that is what makes it an oracle rather than a second opinion. **Two implementations agreeing prove nothing if both are wrong**, so an accelerated method is pinned against the reference and never against another acceleration.

8.2 Structural invariants, where no oracle reaches

Past the sizes enumeration allows, what remains are properties the model must satisfy whatever the answer is:

- rows of a transition matrix sum to one, and $\mathbf{P}(0) = \mathbf{I}$;
- a reversible model satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$;
- the *pulley principle*: on a reversible model the likelihood is invariant to where the root is placed along a branch, so a rooted computation and its re-rooting must agree exactly;
- a gradient matches central differences, relative to the gradient's own norm—entrywise fails wherever an entry is zero, and an absolute bound does not transfer across data sizes;
- an expectation-maximization iterate increases the likelihood monotonically.

None of these establishes that a method is right. Each establishes that a specific way of being wrong did not happen, which is a weaker and more defensible claim.

8.3 Recovery, which is the acceptance test for a fit

A likelihood that increases proves the optimizer runs, not that the model is right. The acceptance test is to fit data simulated from known parameters and require the intervals to cover the truth at their nominal rate over independent replicates. Two qualifications are load-bearing. Where a model has an exact symmetry—permuting the hidden states of an HMM or the components of a mixture leaves the likelihood unchanged—recovery is stated *up to that symmetry*, and the alignment is part of the test. And coverage is measured as a function of how identifiable the instance is, because a fixture chosen where everything is well separated is a test of the fixture.

8.4 Where the likelihood has no maximum, or no curvature

Two failures are properties of models rather than defects, and both must be handled before they are discovered.

An *unbounded* likelihood has no maximum to find: put a Gaussian component's mean on a single observation and let its variance go to zero and the density there diverges. A fit that converges there was stopped by its starting point or by a floor, and which one must be knowable—so the bound is explicit, derived from the data rather than chosen, and reaching it is a refusal rather than a clamp.

A *flat* likelihood has a maximum that runs to the boundary: an overdispersed count model approaches its Poisson limit as the dispersion grows, and past the point where the overdispersion is smaller than the sampling noise on measuring it, the data cannot tell one value from another. The two failures are mirror images, and they announce themselves differently—the first as a divergence, the second as a singular information matrix—which is why each is measured rather than inferred from the other.

8.5 Agreement across implementations is a tolerance

An accelerated implementation is pinned against the reference within a declared *relative* tolerance, never bitwise. Relative, because a log-likelihood is a sum over sites and an absolute bound fixed at one size does not transfer to another. Keyed on the *lowest* precision in the comparison, because a single-precision device cannot be held to a double-precision bound, and one bound loose enough for single precision would let a broken double-precision backend pass. A discrepancy inside the tolerance is not a defect and is not corrected; one outside it is not a tolerance to loosen. The values themselves live beside the measurements they are derived from, not here.

8.6 Published constants, and asymptotic results

Some claims come from outside entirely: the $(2n-5)!!$ count, Goemans–Williamson's 0.87856 for maximum cut, alpha expansion's factor of two, the $8(\ln k + 2)$ bound on k-means++ seeding. These referee an implementation at any size. **No asymptotic result is testable at the sizes an exact oracle allows**, however: a threshold or a limit holds as the problem grows and says nothing where enumeration can check it. Such a result is reported for orientation and the per-instance property is what is asserted.

9 Relevance to the Roadmap

The three problem classes are not three applications. They are chosen so that each abstraction is tested against something that is not the model it was extracted from: a likelihood engine

justified only by trees would be a phylogenetics library, and an optimizer justified only by an HMM would encode a chain.

The order the milestones take follows the dependency between the properties above. Simulation and its ground truth come first, because every later claim is checked against data whose generating parameters are known. The exact evaluators come next, because they are the oracles the accelerated ones are pinned to. Continuous optimization follows, since recovery is the acceptance test and needs both. Discrete search and learned proposals come last, because their acceptance criterion—reaching the optimum enumeration identifies—only exists once enumeration does.

References

[1] Shun-ichi Amari. *Information Geometry and Its Applications*. Springer, Tokyo, 2016.

[2] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.

[3] Richard E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.

[4] Jim Blandy, Jason Orendorff, and Leonora F. S. Tindall. *Programming Rust: Fast, Safe Systems Development*. O'Reilly Media, 2nd edition, 2021.

[5] Yuri Boykov, Olga Veksler, and Ramin Zabih. Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(11): 1222–1239, 2001.

[6] Randal E. Bryant and David R. O'Hallaron. *Computer Systems: A Programmer's Perspective*. Pearson, 3rd edition, 2015.

[7] Phillip Compeau and Pavel Pevzner. *Bioinformatics Algorithms: An Active Learning Approach*. Active Learning Publishers, 2015.

[8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.

[9] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.

[10] David J. Earl and Michael W. Deem. Parallel tempering: Theory, applications, and new perspectives. *Physical Chemistry Chemical Physics*, 7:3910–3916, 2005.

[11] Robert G. Edwards and Alan D. Sokal. Generalization of the Fortuin–Kasteleyn–Swendsen–Wang representation and Monte Carlo algorithm. *Physical Review D*, 38(6):2009–2012, 1988.

[12] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.

[13] G. David Forney. Codes on graphs: Normal realizations. *IEEE Transactions on Information Theory*, 47(2):520–548, 2001.

[14] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.

[15] Robert G. Gallager. Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962.

[16] Charles J. Geyer. Markov chain Monte Carlo maximum likelihood. In *Computing Science and Statistics: Proceedings of the 23rd Symposium on the Interface*, pages 156–163, 1991.

[17] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.

[18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, Massachusetts, 2016.

[19] D. M. Greig, B. T. Porteous, and A. H. Seheult. Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society: Series B*, 51(2):271–279, 1989.

[20] Wen-mei W. Hwu, David B. Kirk, and Izzat El Hajj. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 4th edition, 2022.

[21] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[22] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.

[23] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.

[24] Maxim Lapan. *Deep Reinforcement Learning Hands-On*. Packt Publishing, 2nd edition, 2020.

[25] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.

[26] Robert C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall, 2008.

[27] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.

[28] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.

[29] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 10th anniversary edition, 2010.

[30] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.

[31] Antonio Ortega. *Introduction to Graph Signal Processing*. Cambridge University Press, Cambridge, 2022.

[32] Lior Pachter and Bernd Sturmfels, editors. *Algebraic Statistics for Computational Biology*. Cambridge University Press, Cambridge, 2005.

[33] Simon J. D. Prince. *Understanding Deep Learning*. MIT Press, Cambridge, Massachusetts, 2023.

[34] Sebastian Raschka. *Build a Large Language Model (From Scratch)*. Manning Publications, 2024.

[35] Tom Richardson and Rüdiger Urbanke. *Modern Coding Theory*. Cambridge University Press, Cambridge, 2008.

[36] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.

[37] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.

[38] Robert H. Swendsen and Jian-Sheng Wang. Replica Monte Carlo simulation of spin-glasses. *Physical Review Letters*, 57(21):2607–2609, 1986.

[39] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in monte carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987.

[40] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. A new class of upper bounds on the log partition function. *IEEE Transactions on Information Theory*, 51(7): 2313–2335, 2005.

[41] Ulli Wolff. Collective monte carlo updating for spin systems. *Physical Review Letters*, 62 (4):361–364, 1989.

[42] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory*, 51(7):2282–2312, 2005.

A Derivations Cited From the Text

Each of these is standard; each is here because a claim in the main text depends on it, and the main text cites rather than derives, for the reader the preface names.

A.1 The Bethe free energy is stationary at a sum-product fixed point

Write the beliefs of a pairwise graph as b_i and b_{ij} and constrain them to be normalized and locally consistent, $\sum_{\sigma_j} b_{ij} = b_i$. Introducing a multiplier $\lambda_{ij}(\sigma_i)$ for each consistency constraint and setting the derivative of Equation 18 to zero gives $b_{ij} \propto \psi_{ij}\, e^{\lambda_{ij}(\sigma_i)+\lambda_{ji}(\sigma_j)}$ and $b_i \propto \psi_i\, e^{\sum_{j\in\partial i} \lambda_{ji}(\sigma_i)/(d_i-1)}$; identifying $e^{\lambda_{ji}(\sigma_i)}$ with $\prod_{l\in\partial i\setminus j} m_{l\to i}(\sigma_i)$ turns the consistency constraint into Equation 17. So a fixed point of belief propagation is a stationary point of the Bethe free energy and conversely; on a tree the Bethe free energy is the exact variational free energy, which is why the fixed point there gives $-\log Z$ [42].

A.2 Detailed balance for a cluster move in a field

In the Edwards–Sokal representation the joint over spins and bonds is $p(\sigma,\omega) \propto \prod_{\langle i,j\rangle:\omega_{ij}=1}(1 - e^{-\beta J_{ij}})\,\delta(\sigma_i,\sigma_j)\prod_{\omega_{ij}=0} e^{-\beta J_{ij}} \prod_i e^{\beta h_i(\sigma_i)}$, whose spin marginal is the Boltzmann distribution of Equation 16. Drawing bonds given spins and then recolouring a bond cluster $\mathcal{C}$ uniformly are both Gibbs steps at zero field, so the move preserves p there. With a field the recolouring is no longer a Gibbs step: the ratio of the target before and after recolouring $\mathcal{C}$ from μ to ν is $\exp(\beta\sum_{i\in\mathcal{C}}[h_i(\nu) - h_i(\mu)])$, the bond configuration being unchanged, and a uniform proposal accepted with the Metropolis probability of Equation 23 satisfies detailed balance with respect to that ratio. Omitting the accept step gives a chain that runs and mixes and converges to the zero-field distribution, which is why the distributional test is run with a field as well as without.

A.3 The exchange ratio in parallel tempering

The replicas' joint target is the product $\prod_r \exp(-\beta_r E(\sigma_r))$. Swapping the configurations of replicas i and j changes it by $\exp(-\beta_i E_j - \beta_j E_i)/\exp(-\beta_i E_i - \beta_j E_j) = \exp[(\beta_i - \beta_j)(E_i - E_j)]$, and the proposal is symmetric, so the Metropolis acceptance is Equation 24. Each replica's marginal under the joint is its own tempered Boltzmann distribution whatever the exchanges do, which is the property the per-replica goodness-of-fit test asserts.

A.4 Density evolution on the erasure channel

On a cycle-free (d_v, d_c) graph the messages arriving at a node are independent, so the erasure probability of a message is a number rather than a distribution and Equation 35 reduces to a scalar recursion. A check-to-bit message is known only if every other incoming bit message is known; a bit-to-check message is erased only if the channel and every other incoming check message are. With x_ℓ the probability that a bit-to-check message is erased after ℓ iterations,

$$x_0 = \epsilon, \qquad x_\ell = \epsilon\big(1 - (1 - x_{\ell-1})^{d_c - 1}\big)^{d_v - 1}, \tag{37}$$

the recursion Richardson and Urbanke [35, Sec. 3.12] derives and names density evolution. The right-hand side is increasing in $x_{\ell-1}$ and in ϵ, so x_ℓ is non-increasing and its limit is the largest fixed point in $[0, \epsilon]$; the threshold $\epsilon^\star$ is the largest ϵ for which that fixed point is zero, found by bisection, and for $(3, 6)$ it is 0.4294, with 0.3834 for $(4, 8)$ and 0.5176 for $(3, 5)$. A finite code is not cycle-free, but the neighbourhood of a bit to depth ℓ is a tree with probability tending to one as n grows at fixed ℓ, which is why the recursion is the limit of the flooding decoder and why the code at size is held to bracketing $\epsilon^\star$ rather than to a per-draw agreement—the rule `sim/CLAUDE.md` states for every asymptotic result.

A.5 The delta method for an interval at a fit

At a maximum $\hat{\boldsymbol{\theta}}$ of the log-likelihood, the observed information $\mathcal{I} = -\nabla^2 \log p(\mathbf{D} \mid \hat{\boldsymbol{\theta}})$ gives the asymptotic covariance $\Sigma = \mathcal{I}^{-1}$ of $\hat{\boldsymbol{\theta}}$. A constrained parameter $\phi(\boldsymbol{\theta})$ is a smooth function of $\boldsymbol{\theta}$, so to first order $\mathrm{Var}(\phi(\hat{\boldsymbol{\theta}})) \approx J \Sigma J^\top$ with $J = \partial\phi/\partial\boldsymbol{\theta}$ at $\hat{\boldsymbol{\theta}}$. The statement is asymptotic in the sample size and requires $\mathcal{I}$ to be invertible; a singular or ill-conditioned information is a parameter the data does not identify, and the interval is refused rather than reported (Section 8).

B Derivations of the Bounds

The plug-in bound, (6). $\ell^*(\mathcal{T})$ is a maximum over the feasible set $\boldsymbol{\tau} \geq 0$, and $\hat{\boldsymbol{\tau}}$ lies in it by construction: projected gradient descent on the convex least-squares objective clamps every iterate to the non-negative orthant, so the value at $\hat{\boldsymbol{\tau}}$ is one of the values maximized over. Equality holds exactly when $\hat{\boldsymbol{\tau}}$ is a maximizer.

The parsimony bound, (7). Write $a_e = e^{-k t_e/(k-1)} \in [0, 1]$. Each entry of $\mathbf{P}(t_e)$ is affine in a_e and each branch appears exactly once in the product of (14), so the site likelihood is a polynomial of degree at most one in each a_e: multilinear. A multilinear function on a box attains its maximum at a vertex, where each $\mathbf{P}_e$ is either $\mathbf{I}$ (no change permitted) or $\mathbf{J}/k$ (the child's state independent of the parent's, at weight $1/k$). Root the tree at leaf 1, which the pulley principle permits for a reversible model. At a vertex with cut set C (the branches carrying $\mathbf{J}/k$), the site likelihood is a sum over ancestral assignments constant on each component of $\mathcal{T} \setminus C$: $\pi(x_{1,s})\, k^{-|C|}\, k^{\ell'}$, with ℓ' the number of components containing no leaf (each contributes a free sum over k states), provided every leafy component is monochromatic, and zero otherwise. Merging each leafless component into a neighbour across one of its boundary branches removes

one cut and one leafless component at a time and leaves a valid partition with $|C| - \ell'$ cuts, all of whose components are leafy and monochromatic; assigning each its leaf state gives an ancestral labelling with at most $|C| - \ell'$ changes, so $F_s \le |C| - \ell'$ by minimality of the Fitch score. Hence every vertex value is at most $\pi(x_{1,s})\, k^{-F_s}$, and so is the maximum over the box. Summing over sites gives (7); the regression suite enumerates the $2^{|E|}$ vertices at five taxa and finds the bound attained on every site. The entrywise limit $\sup_t \mathbf{P}_{ij}(t)$, one on the diagonal and $1/k$ off it, gives a bound this one dominates, because that matrix is not row-stochastic and the polynomial is increasing in every entry.

The mean-field bound, (8)**.** For any distribution q on configurations, $\log Z = \mathbb{E}_q[-H] + H(q) + \mathrm{KL}(q \,\|\, p)$, and the divergence is non-negative (Gibbs' inequality). Restricting q to products and evaluating the expectation gives the right-hand side; the fixed-point iteration $q_i \propto \exp(h_i + \sum_{j \in \partial i} J_{ij} q_j)$ ascends it coordinate-wise, and the bound holds at every iterate, so a truncated iteration is a looser bound and never a wrong one [25, 27].

The spanning-tree bound, (9)**.** $\log Z(\theta)$ is the log-normalizer of an exponential family and therefore convex in θ. For trees T with weights $\rho(T)$ and parameters $\theta(T)$ with $\sum_T \rho(T)\theta(T) = \theta$, Jensen gives $\log Z(\theta) \le \sum_T \rho(T) \log Z(\theta(T))$. The implementation takes one tree per edge, built by breadth-first search from that edge so it contains it, weighted uniformly; an edge appearing in a fraction ρ_e of the trees carries J_e/ρ_e on each tree containing it and 0 elsewhere, so the weighted mean is J_e, and the fields are shared. Each $\log Z(\theta(T))$ is exact by the leaf-to-root recursion, in the log domain. The bound is tight when the graph is a tree [40].

The ground-state bracket, (10)**.** $Z(\beta) = \sum_\sigma e^{-\beta H(\sigma)}$ has k^N terms, each at most $e^{-\beta E_{\min}}$ and at least one equal to it, so $-\beta E_{\min} \le \log Z(\beta) \le N \log k - \beta E_{\min}$. Combining with $L \le \log Z(\beta) \le U$ and dividing by $\beta > 0$ gives the bracket; as β grows the $N \log k$ term is divided away, until the bounds on $\log Z$ themselves loosen.

C Reference Taxonomy

Grouped as the repository groups them, so one routing table serves the code and the documents alike.

Infrastructure (build, structure, and speed). Software craft: Martin [26], Blandy et al. [4]. Systems and hardware, including memory hierarchies and parallel kernels: Bryant and O'Hallaron [6], Hwu et al. [20].

Optimization (discrete and continuous). Algorithms and discrete mathematics: Cormen et al. [8], Rosen [36]. Numerical optimization: Nocedal and Wright [30]. Probabilistic inference and message passing: MacKay [25], Koller and Friedman [22], Frey [14], Ortega [31]. Statistical physics and Monte Carlo: Mézard and Montanari [27], Newman and Barkema [28]. Information geometry: Amari [1]. Learning and reinforcement learning: Goodfellow et al. [18], Prince [33], Sutton and Barto [37], Lapan [24], Raschka [34].

Application (the science). Phylogenetics: Felsenstein [12], Durbin et al. [9], Compeau and Pevzner [7], Pachter and Sturmfels [32]. Information and coding: Blahut [3], Gallager [15], Richardson and Urbanke [35], with Nielsen and Chuang [29] as background only.